PYTHON PROGRAMMING		Semester	I/II
Course Code	**1BPLC105B/205B**	CIE Marks	50
Teaching Hours/Week (L:T:P: S)	3:0:2:0	SEE Marks	50
Total Hours of Pedagogy (Theory and Lab hours)	40 + 24 (Practical)	Total Marks	100
Credits	4	Exam Hours	3
Examination type (SEE)	Theory		

Course outcome (Course Skill Set)

At the end of the course, the student will be able to:

CO1: Develop scripts using primitive language constructs of python.
CO2: Identify the methods to manipulate primitive python data structures. CO3: Make use of Python standard libraries for programming.
CO4: Build scripts for performing file operations.
CO5: Illustrate the concepts of Object-Oriented Programming as used in Python.

Module-1

The way of the program: The Python programming language, what is a program? What is debugging? Syntax errors, Runtime errors, Semantic errors, Experimental debugging.

Variables, Expressions and Statements: Values and data types, Variables, Variable names and keywords, Statements, Evaluating expressions, Operators and operands, Type converter functions, Order of operations, Operations on strings, Input, Composition, The modulus operator.

Iteration: Assignment, Updating variables, the for loop, the while statement, The Collatz 3n + 1 sequence, tables, two-dimensional tables, break statement, continue statement, paired data, Nested Loops for Nested Data.

Functions: Functions with arguments and return values.

Chapters: 1.1-1.7, 2.1-2.12, 3.3, 4.4, 4.5

Number of Hours:8

Module-2

Strings: Working with strings as single things, working with the parts of a string, Length, Traversal and the for loop, Slices, String comparison, Strings are immutable, the in and not in operators, A find function, Looping and counting, Optional parameters, The built-in find method, The split method, Cleaning up your strings, The string format method.

Tuples: Tuples are used for grouping data, Tuple assignment, Tuples as return values, Composability of Data Structures.

Lists: List values, accessing elements, List length, List membership, List operations, List slices, Lists are mutable, List deletion, Objects and references, Aliasing, cloning lists, Lists and for loops, List parameters, List methods, Pure functions and modifiers, Functions that produce lists, Strings and lists, list and range, Nested lists, Matrices.

Chapter: 5.1, 5.2, 5.3

Number of Hours: 8

Module-3

Dictionaries: Dictionary operations, dictionary methods, aliasing and copying.

Numpy: About, Shape, Slicing, masking, Broadcasting, dtype.

Files: About files, writing our first file, reading a file line-at-a-time, turning a file into a list of lines, Reading the whole file at once, working with binary files, Directories, fetching something from the Web.

Chapter: 5.4, 6.1-6.5, 7.1-7.8

Number of Hours:8

Module-4

This document is available on studocu

Downloaded by Praanesh Raghavendra (praan.r@gmail.com)

Modules: Random numbers, the time module, the math module, creating your own modules, Namespaces, Scope and lookup rules, Attributes and the dot Operator, Three import statement variants.
Mutable versus immutable and aliasing
Object oriented programming: Classes and Objects — The Basics, Attributes, Adding methods to our class, Instances as arguments and parameters, Converting an instance to a string, Instances as return values.

Chapter: 8.1-8.8, 9.1, 11.1

Number of Hours: 8

Module-5

Object oriented programming: Objects are mutable, Sameness, Copying.
Inheritance: Pure functions ,Modifiers, Generalization, Operator Overloading, Polymorphism.
Exceptions: Catching Exceptions, Raising your own exceptions.

Chapter: 11.2.2-11.2.4, 11.3.2-11.3.9, 12.1, 12.2

Number of Hours:8

PRACTICAL COMPONENTS OF IPCC

PART – A: FIXED SET OF EXPERIMENTS

1. a. Develop a python program to read 2 numbers from the keyboard and perform the basic arithmetic operations based on the choice. (1-Add, 2-Subtract, 3-Multiply, 4-Divide).
   b. Develop a program to read the name and year of birth of a person. Display whether the person is a senior citizen or not.
2. a. Develop a program to generate Fibonacci sequence of length (N). Read N from the console.
   b. Write a python program to create a list and perform the following operations
   - Inserting an element
   - Removing an element
   - Appending an element
   - Displaying the length of the list
   - Popping an element
   - Clearing the list
3. a. Read N numbers from the console and create a list. Develop a program to print mean, variance and standard deviation with suitable messages.
   b. Read a multi-digit number (as chars) from the console. Develop a program to print the frequency of each digit with a suitable message.
4. Develop a program to print 10 most frequently appearing words in a text file. [Hint: Use a dictionary with distinct words and their frequency of occurrences. Sort the dictionary in the reverse order of frequency and display the dictionary slice of the first 10 items.
5. Develop a program to read 6 subject marks from the keyboard for a student. Generate a report that displays the marks from the highest to the lowest score attained by the student. [Read the marks into a 1-Dimesional array and sort using the Bubble Sort technique].
6. Develop a program to sort the contents of a text file and write the sorted contents into a separate text file. [Hint: Use string methods strip(), len(), list methods sort(), append(), and file methods open(), readlines(), and write()].

Downloaded by Praanesh Raghavendra (praan.r@gmail.com)

7. Develop a function named DivExp which takes TWO parameters a, b, and returns a value c (c=a/b). Write a suitable assertion for a>0 in the function DivExp and raise an exception for when b=0. Develop a suitable program that reads two console values and calls the function DivExp.
8. Define a function that takes TWO objects representing complex numbers and returns a new complex number with the sum of two complex numbers. Define a suitable class 'Complex' to represent the complex number. Develop a program to read N (N >=2) complex numbers and to compute the addition of N complex numbers.
9. Text Analysis Tool: Build a tool that analyses a paragraph: frequency of each word, longest word, number of sentences, etc.
10. Develop Data Summary Generator: Read a CSV file (like COVID data or weather stats), convert to dictionary form, and allow the user to run summary queries: max, min, average by column.
11. Develop Student Grade Tracker: Accept multiple students' names and marks. Store them in a list of tuples or dictionaries. Display summary reports (average, topper, etc.).
12. Develop a program to display contents of a folder recursively (Directory) having sub-folders and files (name and type).

Experiments:

1. **a. Develop a python program to read 2 numbers from the keyboard and perform the basic arithmetic operations based on the choice. (1-Add, 2-Subtract, 3-Multiply, 4-Divide).**
   b. Develop a program to read the name and year of birth of a person. Display whether the person is a senior citizen or not.

a. Develop a python program to read 2 numbers from the keyboard and perform the basic arithmetic operations based on the choice. (1-Add, 2-Subtract, 3-Multiply, 4-Divide).

```
# Read two numbers
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

# Display menu
print("\nChoose operation:")
print("1 - Add")
print("2 - Subtract")
print("3 - Multiply")
print("4 - Divide")

choice = int(input("Enter your choice (1-4): "))

# Perform operation
if choice == 1:
print("Result:", num1 + num2)
elif choice == 2:
print("Result:", num1 - num2)
elif choice == 3:
print("Result:", num1 * num2)
elif choice == 4:
if num2 != 0:
print("Result:", num1 / num2)
else:
print("Error: Division by zero is not allowed.")
else:
print("Invalid choice.")
```

Output:

b. Develop a program to read the name and year of birth of a person. Display whether the person is a senior citizen or not.

```
# Read input
name = input("Enter your name: ")
year_of_birth = int(input("Enter your year of birth: "))

# Calculate age
from datetime import date
current_year = date.today().year
age = current_year - year_of_birth

# Check senior citizen (assuming 60 years and above is senior)
print("\nHello,", name)
if age >= 60:
    print("You are a senior citizen.")
else:
    print("You are not a senior citizen.")
```

Output:

2. a. Develop a program to generate Fibonacci sequence of length (N). Read N from the console.

b. Write a python program to create a list and perform the following operations

- **a. Inserting an element**
- **b. Removing an element**
- **c. Appending an element**
- **d. Displaying the length of the list**
- **e. Popping an element**
- **f. Clearing the list**

a. Develop a program to generate Fibonacci sequence of length (N). Read N from the console.

```
N = int(input("Enter the length of Fibonacci sequence: "))
# First two Fibonacci numbers
fibo = [0, 1]
# Generate sequence
for i in range(2, N):
    fibo.append(fibo[i-1] + fibo[i-2])
# Handle small N
if N == 1:
    fibo = [0]
elif N == 0:
    fibo = []
print("Fibonacci sequence of length", N, "is:", fibo)
```

Output:

b. Write a python program to create a list and perform the following operations

a. **Inserting an element**
b. **Removing an element**
c. **Appending an element**
d. **Displaying the length of the list**
e. **Popping an element**
f. **Clearing the list**

```
# Initial list
my_list = []
while True:
    print("\nMenu:")
    print("1. Insert an element")
    print("2. Remove an element")
    print("3. Append an element")
    print("4. Display length of the list")
    print("5. Pop an element")
    print("6. Clear the list")
    print("7. Display the list")
    print("8. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        element = input("Enter element to insert: ")
        pos = int(input("Enter position (index): "))
        my_list.insert(pos, element)
        print("Updated list:", my_list)
    elif choice == 2:
        element = input("Enter element to remove: ")
        if element in my_list:
            my_list.remove(element)
            print("Updated list:", my_list)
        else:
            print("Element not found.")
    elif choice == 3:
        element = input("Enter element to append: ")
        my_list.append(element)
        print("Updated list:", my_list)
    elif choice == 4:
        print("Length of list:", len(my_list))
    elif choice == 5:
        if len(my_list) > 0:
            popped = my_list.pop()
            print("Popped element:", popped)
            print("Updated list:", my_list)
        else:
            print("List is empty, nothing to pop.")
    elif choice == 6:
        my_list.clear()
        print("List cleared:", my_list)
```

```
    elif choice == 7:
        print("Current list:", my_list)
    elif choice == 8:
        print("Exiting program.")
        break
    else:
        print("Invalid choice. Please try again.")
```

Output:

3. **a. Read N numbers from the console and create a list. Develop a program to print mean, variance and standard deviation with suitable messages.**

   b. Read a multi-digit number (as chars) from the console. Develop a program to print the frequency of each digit with a suitable message.

a. Read N numbers from the console and create a list. Develop a program to print mean, variance and standard deviation with suitable messages.

```
import math

# Read N numbers
N = int(input("Enter how many numbers you want: "))
numbers = []

for i in range(N):
    num = float(input(f"Enter number {i+1}: "))
    numbers.append(num)

# Mean
mean = sum(numbers) / N

# Variance
variance = sum((x - mean) ** 2 for x in numbers) / N

# Standard Deviation
std_dev = math.sqrt(variance)

# Output
print("\nResults:")
print("Numbers:", numbers)
print("Mean =", mean)
print("Variance =", variance)
print("Standard Deviation =", std_dev)
```

Output

b. Read a multi-digit number (as chars) from the console. Develop a program to print the frequency of each digit with a suitable message.

```
num_str = input("Enter a multi-digit number: ")

# Dictionary to store digit frequencies
freq = {}

for ch in num_str:
    if ch.isdigit():
        freq[ch] = freq.get(ch, 0) + 1

# Display result
print("\nDigit Frequencies:")
for digit in sorted(freq.keys()):
    print(f"Digit {digit} appears {freq[digit]} time(s).")
```

Output:

4. **Develop a program to print 10 most frequently appearing words in a text file. [Hint: Use a dictionary with distinct words and their frequency of occurrences. Sort the dictionary in the reverse order of frequency and display the dictionary slice of the first 10 items.**

```
# Function to count word frequencies
def word_frequency(filename):
    try:
        with open(filename, "r") as file:
            text = file.read().lower()  # Convert to lowercase for uniformity

        # Split text into words (basic split by whitespace & punctuation handling)
        words = []
        for word in text.split():
            word = word.strip(".,!?;:\"'()[]{}")  # remove punctuation
            if word:  # avoid empty strings
                words.append(word)

        # Count frequency using dictionary
        freq_dict = {}
        for word in words:
            freq_dict[word] = freq_dict.get(word, 0) + 1

        # Sort dictionary by frequency (descending)
        sorted_words = sorted(freq_dict.items(), key=lambda x: x[1], reverse=True)

        # Display top 10 words
        print("Top 10 most frequent words:")
        for word, count in sorted_words[:10]:
            print(f"{word} → {count} times")

    except FileNotFoundError:
        print("Error: File not found. Please check the filename.")

# --- Main Program ---
filename = input("Enter the filename (with extension): ")
word_frequency(filename)
```

Output

5. **Develop a program to read 6 subject marks from the keyboard for a student. Generate a report that displays the marks from the highest to the lowest score attained by the student. [Read the marks into a 1-Dimesional array and sort using the Bubble Sort technique].**

```
# Program to read 6 subject marks and display them sorted (highest to lowest)
# using Bubble Sort

# Read marks into a list
marks = []
print("Enter marks for 6 subjects:")
for i in range(6):
    mark = int(input(f"Subject {i+1}: "))
    marks.append(mark)

# Bubble Sort (descending order)
n = len(marks)
for i in range(n - 1):
    for j in range(n - i - 1):
        if marks[j] < marks[j + 1]:  # swap if next is greater
            marks[j], marks[j + 1] = marks[j + 1], marks[j]

# Display sorted marks
print("\nReport: Marks from highest to lowest")
for idx, mark in enumerate(marks, start=1):
    print(f"{idx}. {mark}")
```

6. **Develop a program to sort the contents of a text file and write the sorted contents into a separate text file. [Hint: Use string methods strip(), len(), list methods sort(), append(), and file methods open(), readlines(), and write()].**

```
# Open the input file in read mode
with open("input.txt", "r") as infile:
    # Read all lines into a list
    lines = infile.readlines()

# Remove extra whitespace/newlines
cleaned_lines = []
for line in lines:
    cleaned_lines.append(line.strip())

# Sort the lines
cleaned_lines.sort()

# Open the output file in write mode
with open("sorted_output.txt", "w") as outfile:
    for line in cleaned_lines:
        outfile.write(line + "\n")

print("File sorted successfully. Check 'sorted_output.txt'")
```

Output:

7. **Develop a function named DivExp which takes TWO parameters a, b, and returns a value c (c=a/b). Write a suitable assertion for a>0 in the function DivExp and raise an exception for when b=0. Develop a suitable program that reads two console values and calls the function DivExp.**

```
def DivExp(a, b):
    # Assertion for a > 0
    assert a > 0, "Assertion failed: a must be greater than 0"
    # Raise exception if b is 0
    if b == 0:
        raise ZeroDivisionError("Division by zero is not allowed")
    # Perform division
    c = a / b
    return c
# Main program
try:
    # Read values from user
    a = float(input("Enter value for a: "))
    b = float(input("Enter value for b: "))
    result = DivExp(a, b)
    print("Result of a/b =", result)
except AssertionError as ae:
    print("Error:", ae)
except ZeroDivisionError as zde:
    print("Error:", zde)
except Exception as e:
    print("Unexpected Error:", e)
```

Output:

8. **Define a function that takes TWO objects representing complex numbers and returns a new complex number with the sum of two complex numbers. Define a suitable class 'Complex' to represent the complex number. Develop a program to read N (N >=2) complex numbers and to compute the addition of N complex numbers.**

```
class Complex:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag

    # String representation
    def __str__(self):
        if self.imag >= 0:
            return f"{self.real} + {self.imag}i"
        else:
            return f"{self.real} - {abs(self.imag)}i"

  def add_complex(c1, c2):
  # Function to add two complex numbers
      return Complex(c1.real + c2.real, c1.imag + c2.imag)

# Main program
N = int(input("Enter the number of complex numbers (N >= 2): "))

if N < 2:
    print("N must be at least 2")
else:
    print("Enter complex numbers (real and imaginary parts):")

    # Read first complex number
    real = float(input("Enter real part of number 1: "))
    imag = float(input("Enter imaginary part of number 1: "))
    result = Complex(real, imag)

    # Read remaining N-1 complex numbers and add them
    for i in range(2, N + 1):
        real = float(input(f"Enter real part of number {i}: "))
        imag = float(input(f"Enter imaginary part of number {i}: "))
        c = Complex(real, imag)
        result = add_complex(result, c)
    print("\nSum of all complex numbers =", result)
```

Output:

9. **Text Analysis Tool: Build a tool that analyses a paragraph: frequency of each word, longest word, number of sentences, etc.**

```
import string

def text_analysis(paragraph):
    # Remove punctuation for word analysis
    translator = str.maketrans("", "", string.punctuation)
    cleaned_text = paragraph.translate(translator)

    # Split into words
    words = cleaned_text.split()
    word_count = len(words)

    # Frequency of each word
    freq = {}
    for word in words:
        word_lower = word.lower()
        freq[word_lower] = freq.get(word_lower, 0) + 1

    # Longest word
    longest_word = max(words, key=len) if words else ""

    # Count sentences (basic: split by ., ?, !)
    sentences = [s for s in paragraph.replace("!", ".").replace("?", ".").split(".") if s.strip()]
    sentence_count = len(sentences)

    # Average word length
    avg_word_len = sum(len(word) for word in words) / word_count if word_count > 0 else 0

    # Return results
    return {
        "word_count": word_count,
        "sentence_count": sentence_count,
        "longest_word": longest_word,
        "frequency": freq,
        "average_word_length": round(avg_word_len, 2)
    }

# Main program
paragraph = input("Enter a paragraph:\n")

result = text_analysis(paragraph)

print("\n--- Text Analysis Report ---")
print("Number of words:", result["word_count"])
print("Number of sentences:", result["sentence_count"])
print("Longest word:", result["longest_word"])
print("Average word length:", result["average_word_length"])
print("\nWord Frequency:")
for word, count in result["frequency"].items():
    print(f"{word}: {count}")
```

Output:

10. Develop Data Summary Generator: Read a CSV file (like COVID data or weather stats), convert to dictionary form, and allow the user to run summary queries: max, min, average by column.

```
import csv

def load_csv(filename):
    #Reads CSV into a list of dictionaries
    with open(filename, "r", newline="") as f:
        reader = csv.DictReader(f)
        data = [row for row in reader]
    return data

def convert_to_float(value):
    #Try converting string to float, else return None

    try:
        return float(value)
    except ValueError:
        return None

def summarize(data, column, operation):
    #Perform summary operation on a column
    values = [convert_to_float(row[column]) for row in data if convert_to_float(row[column]) is
not None]

    if not values:
        return f"No numeric data found in column '{column}'"

    if operation == "max":
        return max(values)
    elif operation == "min":
        return min(values)
    elif operation == "average":
        return sum(values) / len(values)
    else:
        return "Invalid operation"

# Main program
filename = input("Enter CSV filename (with .csv extension): ")

try:
    data = load_csv(filename)
    if not data:
        print("CSV file is empty or invalid.")
    else:
        print("\nAvailable columns:", list(data[0].keys()))

        while True:
            column = input("\nEnter column name for summary (or 'exit' to quit): ")
            if column.lower() == "exit":
                break
```

```
        operation = input("Enter operation (max/min/average): ").lower()
        result = summarize(data, column, operation)
        print(f"{operation.upper()} of '{column}': {result}")

except FileNotFoundError:
    print("Error: File not found.")
```

Output:

11. Develop Student Grade Tracker: Accept multiple students' names and marks. Store them in a list of tuples or dictionaries. Display summary reports (average, topper, etc.).

```
# Initialize an empty list to store student data
students = []

# Read student data
while True:
    name = input("Enter student name (or 'done' to finish): ")
    if name.lower() == "done":
        break

    try:
        marks = float(input(f"Enter marks for {name}: "))
        students.append({"name": name, "marks": marks})
    except ValueError:
        print("Invalid marks. Please enter a numeric value.")

# Check if we have students
if not students:
    print("No student data entered.")
else:
    # Calculate average marks
    total_marks = sum(student["marks"] for student in students)
    average_marks = total_marks / len(students)

    # Find topper
    topper = max(students, key=lambda x: x["marks"])
    lowest = min(students, key=lambda x: x["marks"])

    # Display summary report
    print("\n--- Student Grade Summary ---")
    print(f"Number of students: {len(students)}")
    print(f"Average marks: {average_marks:.2f}")
    print(f"Topper: {topper['name']} with {topper['marks']} marks")
    print(f"Lowest scorer: {lowest['name']} with {lowest['marks']} marks")

    print("\nAll Students:")
    for student in students:
        print(f"{student['name']}: {student['marks']}")
```

Output:

12. Develop a program to display contents of a folder recursively (Directory) having sub-folders and files (name and type).

```
import os

def display_directory(path, indent=0):
    #Recursively display folder contents with type
    if not os.path.exists(path):
        print("Path does not exist.")
        return

    for item in os.listdir(path):
        item_path = os.path.join(path, item)
        if os.path.isdir(item_path):
            print("  " * indent + f"[DIR]  {item}")
            display_directory(item_path, indent + 1) # Recursive call
        else:
            print("  " * indent + f"[FILE] {item}")


# Main program
folder_path = input("Enter folder path to display: ")
print(f"\nContents of '{folder_path}':\n")
display_directory(folder_path)
```

Output: